VoxGuard AI — Complete Project Documentation

Project title: VoxGuard AI — Robust Detection of AI-Generated Speech Using Deep Learning and Audio Forensics
Project type: Final-year CSE (AI & ML) application and research prototype
Documentation date: 26 October 2026
Repository: https://github.com/mohitmjm/VoxGuardAI
Development branch: feature/voxguard-ai

1. Executive summary

VoxGuard AI is a working web application that processes an uploaded voice recording using a genuine pretrained AASIST speech anti-spoofing network. It provides audio playback, an amplitude waveform, an uncalibrated model score, window-level scores, processing metadata and an explanation of limitations. React provides the interface, FastAPI handles uploads and conversion, FFmpeg decodes audio, and PyTorch runs the neural network locally. Inference requires no subscription, paid model API or API key.

The application pipeline works. Its scientific reliability on modern voice generators has not been established. By default, every completed analysis is labeled **Uncertain**, regardless of score, because no defensible deployment thresholds have been fitted and approved. This is a deliberate abstention policy, not a random prediction or a failed model invocation. An optional validated policy can enable Likely Human-Recorded and Likely AI-Generated decisions within a documented domain.

A small measured pilot exists: three human recordings and three classical Flite TTS recordings. This is an engineering and exploratory result, not representative evidence of accuracy. Modern neural TTS, cloned voices, Hindi and Marathi remain evaluation goals. The current operational choice is to demonstrate the frontend and backend locally on the user's laptop. The user has separately deployed the frontend to Vercel; a publicly hosted inference backend has not been established.

2. Problem statement and motivation

Modern text-to-speech and voice-conversion systems can imitate natural speech and speaker identity. This creates risks including impersonation, misleading media, social engineering and unauthorized voice cloning. Human listeners cannot consistently identify every synthetic recording, particularly after compression or background noise.

The project investigates whether learned acoustic patterns can help distinguish genuine and synthetic speech. It does not establish who spoke, whether the words are true, whether a speaker consented, or whether the recording has been edited in some other way. An AI-generated video may contain human-recorded speech; synthetic visuals alone do not label its audio as synthetic.

The central engineering problem is delivering real model inference through a safe, usable upload application. The research problem is determining how well that detector transfers beyond its original training domain and whether hybrid representations improve performance under controlled evaluation.

3. Objectives, scope and success criteria

The implemented application objectives are to accept supported audio, validate it, convert it consistently, invoke genuine pretrained weights, return structured results and display interpretable scores without invented confidence. The system also supplies reproducible evaluation tools and keeps experiments separate from application inference.

Application success means that a real recording produces a real, finite model score; invalid input produces a meaningful error; temporary audio is deleted; the interface displays the response correctly; and local setup is documented. Research success requires separate evidence: legitimate datasets, audited partitions, independent test recordings, frozen thresholds, measured errors and reproducible comparisons. Successful application operation does not establish research success.

This version does not implement authentication, a user database, permanent upload history, streaming detection, video-forensics analysis, guaranteed authenticity verification, or a production-trained hybrid classifier.

4. Users and practical uses

The primary users are students, supervisors and researchers demonstrating or studying speech anti-spoofing. A user may inspect a consented microphone recording, authorized TTS output or voice-cloning example. The results can support exploratory analysis and dataset evaluation. They must not serve as sole evidence for accusing a person, rejecting a claim, or verifying identity.

5. Technology stack

Layer	Technology	Responsibility
Interface	React 19, TypeScript	Upload workflow, results and accessible controls
Frontend tooling	Vite 6, Tailwind CSS 4	Development server, build and styling
Icons and waveform	Lucide React; browser Canvas and AudioContext	UI icons and amplitude visualization
HTTP service	FastAPI, Uvicorn, Pydantic	Endpoints, multipart handling and response validation
Audio conversion	FFmpeg and FFprobe	Content probing, mono conversion and resampling
Deep learning	PyTorch 2.6.0	Genuine pretrained AASIST inference
Numerical/audio libraries	NumPy 1.26.4, SoundFile 0.13.1	Arrays and research audio utilities
Testing	Pytest, HTTPX, Vitest, Testing Library	Backend, model, research and UI verification
Optional browser checks	Playwright Core and Chromium	Actual upload-to-result integration
Optional research encoder	TorchAudio 2.6.0 WAV2VEC2_BASE	Frozen learned features for experiments

Python 3.11 or 3.12 is the documented baseline; Python 3.12 and CPU PyTorch 2.6 were exercised in the development environment. Use a compatible Node.js release: Node 20.19+ or Node 22.12+ is documented for local use. A current supported Node 22 release is suitable for deployment. Hardware performance depends on recording length and available CPU; there is no guaranteed latency or universal minimum-memory claim.

6. Repository organization

```
VoxGuardAI/
  frontend/                 React application, tests and Vercel configuration
  backend/                  FastAPI app, response schemas, research report loading
  ml/                       AASIST service, downloader, features and policy loader
    vendor/                 Upstream AASIST architecture, license and notice
    checkpoints/            Downloaded weights; excluded from Git
  research/                 Dataset protocols, evaluation and policy tools
    datasets/               Access, rights, manifests and import tooling
    validation/             Audited validation runner and uncertainty tools
    results/                Measured small-pilot reports and portable scores
    paper/                  Manuscript, references, figures and draft PDF
  experiments/              Robustness and frozen-feature comparison experiments
  tests/                    API, model, schema, policy and browser integration
  scripts/                  Setup, preflight, verification and performance utilities
  docs/                     Setup, architecture, status and detailed documentation
  .env.example              Example environment values
  docker-compose.yml        Optional container configuration
  README.md                 Entry-point instructions
```

The frontend owns presentation; the backend owns transport, validation and decoding; the ML service owns model loading and score computation. Research and experiment code must not silently alter stable production inference.

7. Architecture and data flow

```
User chooses audio in browser
          |
          v
React: playback + waveform + multipart upload
          |
          v
FastAPI: bounds, extension, concurrency and temporary storage
          |
          v
FFprobe: container/stream checks
          |
          v
FFmpeg: mono float PCM at 16 kHz
          |
          v
ModelService: windowing -> official AASIST -> softmax scores
          |
          v
Duration-weighted aggregation + optional validated decision policy
          |
          v
Structured JSON -> React report, timeline and JSON export
          |
          v
API temporary files removed, including error paths
```

During local development, Vite forwards `/api` requests to the backend on port 8000. With separate hosting, `VITE_API_BASE_URL` determines the backend origin embedded in the frontend build. Backend CORS settings must allow that frontend origin.

8. Complete user workflow

1. Open the local frontend and check the backend/model readiness message.
2. Choose or drag a WAV, MP3, M4A or FLAC recording into the upload area.
3. Review its filename, size, audio playback and browser-generated waveform.
4. Select Analyze recording. The frontend submits multipart field `file`.
5. The backend validates, decodes, resamples and runs the model.
6. The interface shows a classification, raw model score, duration, sample rate and model-call time.
7. Inspect segment scores and click a segment to seek the audio player.
8. Expand processing information and interpretation limitations or download the JSON report.
9. Remove or replace the recording to analyze another file.

Canceling a request stops the browser waiting for it. It does not guarantee immediate interruption of native model computation on the server. The server completes cleanup when processing exits. A waveform shows amplitude, not proof of synthetic speech. Browser codec support may prevent waveform rendering while server decoding remains available.

9. Audio validation and preprocessing

Default upload size is 20 MiB (20 × 1,024 × 1,024 bytes), displayed as 20 MB in the interface. Default duration is at most 120 seconds and at least 0.5 seconds. A 13-minute recording can satisfy the byte limit and still fail the duration limit.

The backend validates the extension against WAV, MP3, M4A and FLAC and separately checks the detected container with FFprobe. It expects one audio stream and rejects video streams except attached cover artwork. It rejects empty, corrupt, excessively long, nonfinite and essentially silent decoded signals. Content validation reduces risk; it does not constitute a complete adversarial-media security guarantee.

FFmpeg is invoked with an argument list rather than a shell. Network protocols are restricted to local file/pipe access; subprocess calls have timeouts; decoding is duration-bounded. Output is one-channel 16 kHz float32 PCM. The original container, channel count and sample rate are retained as processing metadata. Mono conversion and resampling can affect acoustics and therefore belong in the reproducible inference protocol.

The decoder's duration cap and frontend size checks do not replace server-side limits. The API enforces its own upload limit and serializes inference with a concurrency gate. Temporary directories contain the upload and decoded PCM and are removed after processing, including failures.

10. Genuine pretrained AASIST inference

AASIST means **Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks**. The project uses the official NAVER Clova architecture and pretrained anti-spoofing checkpoint, rather than an unrelated speech-recognition or speaker-identification model.

At a conceptual level, its waveform frontend and residual encoder learn acoustic representations. Spectral and temporal graph-attention components model relationships across frequency-related and time-related features; integrated graph processing and pooling produce an utterance representation for classification. The application imports the upstream architecture from `ml/vendor/aasist.py`. It does not claim to have invented or trained AASIST.

Provenance item	Value
Official source	https://github.com/clovaai/aasist
Frozen upstream revision	a04c9863f63d44471dde8a6abcb3b082b07cd1d1
Model family	AASIST speech anti-spoofing

Provenance item	Value
Training domain documented here	ASVspoof 2019 Logical Access
Code license	MIT; retained upstream license and notice
Checkpoint SHA-256	51d2d9cf0738172f61e2a384ec50a54a55363240f67c971ed55a92435bc1a1c0
Downloader	python -m ml.download_model
Default device	CPU

The downloader obtains weights from an immutable official GitHub revision, verifies the expected SHA-256 and installs them atomically. The digest records agreement with the upstream artifact checked during development; it is not an author signature. Dataset rights are separate from model-code licensing. The loader uses `torch.load(..., weights_only=True)`, checks the checkpoint and loads its state dictionary strictly. Missing or corrupted weights produce an explicit setup error. Predictions are never simulated.

CPU execution is the supported exercised baseline. CUDA is configurable with a compatible PyTorch/CUDA installation but has not been verified in this environment. The neural network runs in evaluation/inference mode. Default CPU thread count is two.

11. Windowing, scores and aggregation

The model expects 64,600 samples per window at 16,000 samples per second, equivalent to 4.0375 seconds. A shorter recording or final tail is repeated until it reaches the expected length, then truncated to exactly 64,600 samples. It is not padded with zeros. This follows the upstream short-input preparation.

Longer recordings are divided into consecutive non-overlapping windows. Each window receives a softmax score for class 0, the spoof class; class 1 is bona fide. If the two output logits are z_spoof and z_human, the reported window score is:

```
s_i = exp(z_spoof) / [exp(z_spoof) + exp(z_human)]
S = sum(d_i × s_i) / sum(d_i)
```

Here d_i is the original duration represented by window i, before repeat padding. Duration weighting prevents a tiny repeated final tail from receiving the same aggregate weight as a full window. This long-recording aggregation is a project application extension and requires evaluation; it is not a claim about the original model's benchmark protocol.

Scores range from zero to one. A higher score is more synthetic-oriented according to this model. The score is not an estimate of what fraction of the recording is synthetic, a calibrated authenticity probability, or a confidence percentage. One second of synthetic speech mixed with human speech is not guaranteed to produce a proportional score.

Segment scores describe responses to fixed windows. They do not locate exact edits, identify a particular generator, or prove that an individual interval is synthetic.

12. Why a high score still displays Uncertain

The default policy abstains from Human/AI decisions until representative validation establishes suitable operating thresholds. Therefore even a score close to one still returns `uncertain`. The lack of a label does not mean the model failed to run.

In the user's demonstration screenshot, the recording was 72.66 seconds long, the displayed aggregate score was approximately 0.852 and the model-call time was approximately 20.06 seconds. These are user-observed application values, not independently reproduced benchmark results. A bar at

85.2% of its range represents score 0.852; it does not mean 85.2% certainty or that 85.2% of the speech is AI-generated. The user's statement that a video was AI-generated is not, by itself, verified audio ground truth.

A reviewed two-threshold policy can classify scores below a low boundary as Human, above a high boundary as AI, and scores in between as Uncertain. Thresholds must be fitted only on validation data, frozen, tested on untouched data and bound to checkpoint/protocol/domain provenance. Such a policy is not probability calibration and cannot automatically verify that an upload belongs to the validated domain. No enabled deployment policy is bundled.

13. API contract

Method and route	Purpose
GET /api/health	Software/model availability, FFmpeg availability and upload limits
GET /api/model/info	Model name, revision, checkpoint digest, device and decision-policy metadata
POST /api/analyze	Multipart audio upload using field `file`; returns analysis JSON
GET /api/research/results	Trusted configured offline research summaries, or not-evaluated state
GET /docs	FastAPI interactive API documentation

The existing backend does not promise a homepage at `/`; use the health endpoint or `/docs` when verifying it. Model availability is distinct from successful inference: a health response alone does not establish checkpoint usability, detector accuracy, or complete readiness.

A successful analysis contains these fields:

Field	Meaning
classification	`human`, `ai` or `uncertain`; default is `uncertain`
score	Finite uncalibrated aggregate score in [0,1]
segments	Ordered start/end times and finite scores for each window
duration_seconds	Decoded original duration
sample_rate	16000
model	Name, provenance, device, policy and other metadata
processing	Original format/channels/rate and converted rate/channels
latency_ms	Model-service call time; conversion is excluded
limitations	Interpretation and domain caveats

Pydantic checks finite scores, valid classifications and ordered segment bounds. Errors use meaningful HTTP statuses, including 413 for excessive upload bytes, 415 for unsupported/mismatched formats, 422 for invalid audio or duration, 429 for an occupied inference gate, 503 for unavailable dependencies/model, and 504 for model deadline expiration. Multipart-envelope timeouts can return 408. Unexpected stack traces and server file paths are not returned to users.

14. Frontend implementation and usability

The application supplies a responsive navy/cyan interface, accessible light/dark modes, upload states, audio playback, waveform rendering, progress indication, cancellation, errors, model readiness, scores, segment seeking and downloadable JSON. The browser reads audio locally for visualization using AudioContext and Canvas; inference is performed by the backend.

The frontend validates basic file properties before upload and also consumes server limits. The backend remains authoritative. The selected file is held by the browser for playback until replacement or page close. Object URLs are released when replaced. The application does not maintain a permanent account-level recording history.

The research dashboard loads only sanitized offline summaries explicitly configured by an operator. It does not calculate a benchmark from arbitrary user uploads or display invented performance figures.

15. Privacy, security and operational bounds

The default API creates temporary files for each request and deletes them in cleanup paths. Uploaded recordings are not used for training. Application logging must not capture recording bodies or sensitive audio content. Browser playback retention is distinct from server storage. Any future provider, reverse proxy, analytics or monitoring configuration needs its own review because it can change retention outside this application's control.

The service restricts supported containers, decoding protocols, request size, duration, subprocess timeouts and concurrent inference. CORS is configured explicitly; it controls browser cross-origin access and is not authentication or a complete abuse-prevention mechanism. The initial MVP has no accounts, authentication, persistent database or distributed job queue.

The inference deadline is cooperative and checked before and after model windows. It cannot interrupt an active native PyTorch operation or initial model load. Hard resource guarantees would need process/container or host-level controls. Public operation also needs appropriate rate limiting, TLS, monitoring and resource isolation. Local development is the intended current demo mode.

16. Configuration

Variable	Default or purpose
VOXGUARD_CORS_ORIGINS	Backend defaults to local Vite origins; comma-separated allowed origins
VOXGUARD_MAX_UPLOAD_MB	20
VOXGUARD_MAX_DURATION_SECONDS	120
VOXGUARD_DEVICE	cpu
VOXGUARD_TORCH_THREADS	2
VOXGUARD_INFERENCE_TIMEOUT_SECONDS	120
VOXGUARD_CHECKPOINT	Optional alternative checkpoint path; same expected digest
VOXGUARD_POLICY_PATH	Optional reviewed domain-specific decision-policy artifact
VOXGUARD_RESEARCH_REPORTS_PATH	Optional trusted sanitized offline research summaries
VITE_API_BASE_URL	Frontend build-time backend origin; empty for local Vite proxy

`.env.example` is a reference, not proof that Python automatically loads an `.env` file. The backend reads process environment variables; set them in PowerShell, the shell or the hosting service before

startup. Vite loads its own supported frontend env files. Never put secrets in frontend variables, because Vite embeds their values in downloadable client code.

17. Windows installation from a clean environment

Install Python 3.11 or 3.12, a compatible Node.js release, Git and FFmpeg. The following commands assume the user's existing checkout location. Adjust that path on another machine. Use ordinary PowerShell; a virtual-environment activation script is not required because commands invoke its Python directly.

```
winget install --id Gyan.FFmpeg --exact
# Close and reopen PowerShell after installation, then verify:
ffmpeg -version
ffprobe -version

cd C:\Users\Mohit\VoxGuardAI-local
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install torch==2.6.0 --index-url
https://download.pytorch.org/whl/cpu
.\.venv\Scripts\python.exe -m pip install -r backend/requirements.txt -r ml/requirements.txt
.\.venv\Scripts\python.exe -m ml.download_model
.\.venv\Scripts\python.exe -m pip check
.\.venv\Scripts\python.exe scripts/preflight.py
```

If only Python 3.11 is installed, substitute `py -3.11`. The setup requires Internet access for packages and the initial official checkpoint download. Once dependencies and weights are present, normal local inference needs no external prediction service. Do not recreate a working environment for every run.

18. Everyday local demonstration

Terminal 1 starts the backend:

```
cd C:\Users\Mohit\VoxGuardAI-local
.\.venv\Scripts\python.exe -m uvicorn backend.app:app --host 127.0.0.1 --port 8000
```

Terminal 2 starts the frontend:

```
cd C:\Users\Mohit\VoxGuardAI-local\frontend
npm ci
npm run dev
```

`npm ci` is needed for initial installation or lockfile changes, not every launch. Open the address Vite prints, usually `http://localhost:5173`. Check `http://127.0.0.1:8000/api/health` or `/docs` if debugging. If Vite uses a different port, ensure the corresponding origin is allowed by the backend. Keep both terminals open during the demonstration and press Ctrl+C to stop each server afterward.

Use a consented human speech clip and an authorized synthetic speech clip, ideally 5–30 seconds long. Explain the actual score, current abstention policy and limitations. Do not claim that two successful examples prove accuracy. Download the report JSON if a record of the demo is needed; retain or share it responsibly.

19. Linux/macOS setup

```
python3 -m venv .venv
.venv/bin/python -m pip install torch==2.6.0 --index-url https://download.pytorch.org/whl/cpu
.venv/bin/python -m pip install -r backend/requirements.txt -r ml/requirements.txt
.venv/bin/python -m ml.download_model
.venv/bin/python scripts/preflight.py
.venv/bin/python -m uvicorn backend.app:app --host 127.0.0.1 --port 8000
# Separate terminal, from the repository:
cd frontend
npm ci
npm run dev
```

Install FFmpeg/FFprobe through the platform's supported package mechanism. On all platforms, run Python module commands from the repository root so local packages resolve correctly.

20. Troubleshooting the demonstrated issues

Symptom	Cause and corrective action
`.venv/bin/python` is not recognized in PowerShell	That is a POSIX path; use `.\.venv\Scripts\python.exe`
Windows virtual-environment Python is not found	Change to the repository folder and ensure the environment was created there
`No module named numpy` or `uvicorn`	Dependency installation did not complete in that environment; resolve installation failure first
Dependency resolver conflict	Use documented Python/PyTorch/NumPy baseline and current requirements; do not select nonexistent versions
FFmpeg and FFprobe must be installed	Install both, reopen the terminal to refresh PATH, verify both commands, restart backend
Error 10048 / port 8000 in use	Stop the earlier backend with Ctrl+C before restarting; identify an unknown listener before terminating it
Recording exceeds duration limit	Upload a shorter recording; bytes and duration are separate checks
Vercel shows backend offline	Static frontend deployment does not host PyTorch inference; use local frontend for the current local demo
Score 0.852 but result Uncertain	Expected default policy: no validated classification thresholds are enabled
Model missing/checksum error	Run the official model downloader from the same environment and check configured checkpoint path
Browser waveform unavailable	Browser codec decoding may be unsupported even when FFmpeg can analyze the upload
Analysis timeout	Try shorter speech and confirm CPU configuration; long clips require multiple model windows

To extract a small speech segment, replace the input path with a real local file:

```
ffmpeg -i "C:\path\to\recording.mp3" -t 20 -ac 1 -ar 16000 "$env:USERPROFILE\Desktop\demo.wav"
```

21. Testing and evidence

The repository contains unit tests, API integration tests, genuine model smoke tests, preprocessing equivalence checks, policy/provenance tests, research numerical tests and frontend component tests. Mock models are used only in isolated tests that need controlled responses; production inference uses real weights. A generated tone can verify numerical operation but cannot establish speech classification quality.

The latest persisted verification record reports **145 Python tests**, **16 frontend tests** and a successful frontend production build in the development environment. These are historical execution results recorded in `docs/VERIFICATION.md`, not a new test run performed while writing this document. Actual Windows interaction shown by the user also demonstrates local end-to-end inference; the complete Windows installation matrix has not been independently tested here.

Recorded Chromium integration used actual AASIST inference on a licensed 14.84-second human recording and a 3.545-second classical Flite recording. Displayed scores matched API responses; report

download, invalid uploads and mobile layout checks passed. This establishes integration, not reliability. Docker build execution was blocked by package-host DNS/network access; CUDA was not tested.

To run the current checks after installing development/research dependencies:

```
.\.venv\Scripts\python.exe -m pip install pytest httpx scikit-learn scipy
.\.venv\Scripts\python.exe -m pip install torchaudio==2.6.0 --index-url
https://download.pytorch.org/whl/cpu
.\.venv\Scripts\python.exe -m pytest tests research ml experiments -q
cd frontend
npm test -- --run
npm run build
```

Some optional model or audio tests depend on locally cached encoder weights or an FFmpeg build containing the optional Flite filter; report skips rather than treating them as passes. Browser reproduction and dependency details are in `docs/SETUP.md` and `docs/VERIFICATION.md`.

22. Measured pilot results and their meaning

The recorded clean pilot used six English recordings: three LibriSpeech-derived human excerpts and three classical Flite TTS utterances. The clean report records AUROC 1.0 and EER 0.0; no validation threshold, binary classification accuracy, precision, recall or F1 was established. Clean model-call latency median was approximately 712.55 ms and p95 approximately 1,119.16 ms on the recorded development run. These timings exclude decoding and are specific to the sample lengths, host and run.

Condition	Underlying recordings	AUROC	EER
Clean	6: 3 human + 3 classical TTS	1.0	0.0
MP3 32 kbps	Same 6, paired	1.0	0.0
MP3 64 kbps	Same 6, paired	1.0	0.0
AAC 64 kbps	Same 6, paired	1.0	0.0
Seeded white noise, 20 dB SNR	Same 6, paired	1.0	0.0
Seeded white noise, 10 dB SNR	Same 6, paired	1.0	0.0

Six conditions produce 36 observations, not 36 independent recordings. The clean row in the standalone clean report and the clean robustness row reuse data and must not be added as independent evidence. Class and recording conditions are strongly confounded; Flite is not a representative modern neural voice generator. Perfect separation in this tiny convenience set cannot support a 100% accuracy claim, broad robustness claim or deployment threshold. See `research/results/pilot_clean.json`, `pilot_robustness.json`, `pilot_scores.csv` and `pilot_provenance.json` for exact records.

A separate recorded mechanical performance observation used an AMD EPYC 9V45 host, Python 3.12.14, PyTorch 2.6 CPU and two threads: a one-second tone had a cold call of 275.74 ms, a warm call of 214.74 ms and process peak resident memory of 476,280 KiB. These are individual mechanical observations, not speech benchmarks or hosting memory guarantees.

23. Research question and dataset requirements

The proposed research title is **Robust Detection of AI-Generated Speech Across Unseen Voice Synthesis Systems Using Hybrid Acoustic Representations**. The hypothesis is that combining pretrained speech embeddings, frequency-domain features and compression-aware training may improve transfer to unfamiliar synthesis systems. No improvement or novelty has yet been demonstrated.

Candidate data include official ASVspoof releases, In-the-Wild, ASVspoof 5, WaveFake and other modern synthetic speech collections. Each release requires source, access, label semantics, license and overlap review. A publicly downloadable file is not automatically licensed for redistribution. The code license does not license underlying voices or datasets.

Attempts to obtain representative requested releases encountered network/proxy restrictions. Rights and metadata for some requested sources remain unverified; zero new representative recordings were acquired in the last dataset sprint. The repository provides access investigations, resumable/partial acquisition and manual import workflows, but does not include a massive corpus or claim successful acquisition. Do not bypass network policy. Preserve official recording IDs, partitions, speaker IDs, source-utterance lineage and generator families.

Hindi and Marathi require suitable licensed data or explicitly consented recordings and authorized synthetic counterparts. Language compatibility is an evaluation goal, not an established feature merely because the decoder can read the file.

24. Leakage prevention and experimental discipline

Training, validation and test partitions must be defined before selection. All derivatives of an original source utterance—including crops, encodings, noise variants and cloned/synthetic derivatives—must remain in the appropriate linked partition. Speaker identity and source identity must be audited separately. A byte hash finds identical content but cannot prove that re-encoded or cropped speech is independent.

Generator-held-out experiments must exclude test synthesis families from both training and validation according to the declared protocol. The existing pretrained model may have unknown exposure to external data; unresolved overlap must be disclosed. Normalize features using training data only. Select hyperparameters, epochs and thresholds using validation data only. Evaluate untouched test data after freezing the configuration.

Record Git revision, model digest, manifest and feature hashes, software versions, hardware, seeds, preprocessing, class counts, language/generator breakdowns and protocol deviations. For repeated robustness conditions, use paired comparisons and speaker/source-aware uncertainty rather than pretending each encoding is independent.

25. Evaluation metrics

AI is the positive class: TP means AI correctly labeled AI; FP means human incorrectly labeled AI; FN means AI incorrectly labeled human; TN means human correctly labeled human.

```
Accuracy  = (TP + TN) / (TP + TN + FP + FN)
Precision = TP / (TP + FP)
Recall    = TP / (TP + FN)
F1        = 2 × Precision × Recall / (Precision + Recall)
FPR       = FP / (FP + TN)
FNR       = FN / (FN + TP)
```

Undefined denominators need explicit treatment in reports. AUROC measures ranking discrimination across operating points. EER is the interpolated operating point where false-positive and false-negative rates coincide; a test-set EER is descriptive, not a test-selected deployment threshold. A binary confusion matrix assumes a frozen binary threshold.

An abstaining classifier requires a two-class by three-decision matrix, coverage, per-class abstention and accepted-only accuracy with explicit denominators. An abstention is not a correct Human/AI decision. Report cold and warm latency, decoding versus model time, median/p95 and hardware. Speaker-cluster bootstrap intervals and valid replicate counts are supported; tiny samples cannot justify reliable population conclusions.

26. Research tools and reproduction

These commands require legitimate data and appropriate installed research dependencies; they do not download or invent a corpus:

```
python -m research.evaluate audit data/manifest.csv --held-out-generators --output
artifacts/audit.json
python -m research.evaluate score data/manifest.csv --output artifacts/scores.csv
python -m research.evaluate fit-threshold data/validation_scores.csv --output
artifacts/threshold.json
python -m research.evaluate report data/test_scores.csv --threshold artifacts/threshold.json
--output artifacts/metrics.json
python -m experiments.robustness data/manifest.csv --output artifacts/robustness.csv
```

Create separate validation/test score files preserving required headers and metadata. The binary threshold artifact above is for research reporting and is not automatically an application decision policy. Omit a generator-held-out claim if trustworthy generator metadata cannot be obtained.

For a reviewed application abstention policy, follow `docs/DECISION_POLICY.md`. The fitter normally requires at least 100 recordings and 10 speakers per class; these are safeguards, not proof of representativeness:

```
python -m research.fit_policy data/validation_scores.csv --checkpoint-sha256
51d2d9cf0738172f61e2a384ec50a54a55363240f67c971ed55a92435bc1a1c0 --domain "Actual validated
recording domain" --output policy.json
python -m research.report_policy data/test_scores.csv --policy policy.json --checkpoint-sha256
51d2d9cf0738172f61e2a384ec50a54a55363240f67c971ed55a92435bc1a1c0 --output
artifacts/test_policy_metrics.json
```

Do not weaken dataset safeguards just to create a demonstration label. Enable a policy only after reviewing untouched-test evidence, then set `VOXGUARD_POLICY_PATH` and restart the backend.

27. Experimental hybrid features and ablations

The optional experimental branch combines a frozen Wav2Vec2 speech encoder with LFCC acoustic summaries. Wav2Vec2 representation pretraining is not spoof-detector training. Its official WAV2VEC2_BASE encoder is a separate approximately 360 MB download. The documented extraction pools final-layer mean/std within fixed four-second windows and aggregates by duration; LFCC mean/std form the acoustic branch. A small classifier head can compare learned-only, acoustic-only and fusion settings.

```
python -m pip install torchaudio==2.6.0 --index-url https://download.pytorch.org/whl/cpu
python -m experiments.extract_features data/manifest.csv --output artifacts/features.npz
python -m experiments.hybrid_evaluate artifacts/features.npz --baseline-scores artifacts/scores.csv
--output artifacts/comparison-seed42 --dataset "Verified dataset release and protocol" --seed 42
```

The comparison matches recording/content/partition identities and uses train-only normalization and validation-only selection. Default data safeguards include minimum class counts per split. Numerical unit fixtures establish software behavior, not a useful trained model. No representative hybrid training, useful hybrid accuracy result or production fusion checkpoint exists. Head-only timing must not be compared with full AASIST end-to-end timing as a speed advantage.

Experiments A–F are: baseline detection; unseen-generator transfer; paired compression; paired noise; learned-plus-acoustic fusion; and component/augmentation ablations. Only the bounded clean/compression/noise convenience pilot has measured findings. Representative baseline transfer, unseen modern generators and empirical hybrid ablations remain pending.

28. Deployment choices and current status

Local operation is the supported current demonstration choice and does not require Docker. The frontend is independently deployable to Vercel with root directory `frontend`, Vite framework, `npm ci`, `npm run build` and output `dist`. The user reported a successful frontend deployment. That deployment alone does not run Python, FFmpeg or AASIST.

A separately hosted backend needs a persistent compatible Python process, PyTorch memory headroom, FFmpeg/FFprobe, writable temporary storage and a public HTTPS endpoint. Set `VITE_API_BASE_URL` to that origin and rebuild/redeploy the frontend; set backend CORS to the exact Vercel origin. No actual public backend URL has been established in this session. Free-host quotas, sleeping behavior, hardware access and costs vary and must be checked on the user's account. The user's Hugging Face interface marked Docker paid; earlier assumptions about free Docker availability did not apply to that account.

A Gradio package was prepared as a separate demo, but the download path failed in the chat UI and it has not been deployed or linked to Vercel. The user chose to stop that hosting route. It must not be reported as a live backend. Optional Docker configuration exists in the repository, but cloud image-building encountered network/DNS restrictions and was not verified. No payment, public backend deployment or always-on availability is claimed.

29. Limitations, failures and future work

The largest limitation is missing representative validation and independent test evidence. Scores can be affected by language, generator, microphone channel, noise, compression, silence and music. No speech-activity detector filters all unsupported content. The original model training domain does not guarantee performance on new neural voices, natural real-world recordings or mixed human/synthetic audio.

The current binary-label abstention is intentional. It makes the prototype less decisive for demonstrations but prevents unsupported authenticity claims. Future work should prioritize licensed data acquisition, leakage audits, held-out generator/language evaluation, calibration where justified, and domain-specific reviewed operating points. Only then should thresholds and performance claims change.

Further engineering work can include better speech/non-speech checks, real-world noise conditions, process-level hard timeouts, authenticated/rate-limited hosting, cold-start management and broader Windows/CUDA deployment testing. Research work can evaluate hybrid features, compression-aware augmentation, multiple seeds, matched costs, ablations and confidence intervals. None is a completed improvement until measured.

30. Final-year report and manuscript readiness

The existing manuscript sources and draft PDF live in `research/paper/`. They discuss the baseline, proposed hybrid study, measured limited pilot, methodology and unfinished comparisons. They are research drafts, not a submission-ready validated contribution.

A final academic submission should include an abstract, problem statement, requirements, literature review, system design, model details, implementation, dataset/ethics statement, reproducible experiments, measured results, error analysis, limitations, conclusion and verified references. Include screenshots of the actual application and distinguish engineering test evidence from detector-quality evaluation. Do not copy upstream benchmark numbers as results achieved by this application or label conventional feature fusion as novel without a supported contribution.

For a viva, the core explanation is: a browser uploads speech; FastAPI validates it; FFmpeg prepares mono 16 kHz audio; genuine pretrained AASIST processes fixed windows; scores are aggregated; React displays model evidence; and the system abstains until validated thresholds support decisions. No training from the user's recordings occurs.

31. Completion matrix

Capability	Status
React upload, player, waveform and report	Implemented and exercised
FastAPI upload and audio validation	Implemented and exercised
Real official AASIST CPU inference	Implemented and exercised
Temporary API upload cleanup	Implemented and tested
Segment scores and JSON export	Implemented and exercised
Human/AI labels from reviewed domain policy	Infrastructure implemented; no deployment policy enabled
Local frontend/backend demonstration	Working; user supplied successful result screenshot
Independent frontend deployment	User-reported Vercel success
Public linked inference backend	Not established; user selected local demo
Research metrics and leakage tools	Implemented; numerical checks available
Tiny six-recording clean/robustness pilot	Measured; nonrepresentative
Modern generators and Hindi/Marathi validation	Pending
Empirical hybrid comparison	Tooling available; representative training/evaluation pending
Docker/CUDA validation	Not established
Research manuscript	Draft; not submission-ready

32. References and primary sources

1. Jung et al., *AASIST: Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks*, arXiv:2110.01200; associated ICASSP 2022 work. https://arxiv.org/abs/2110.01200
2. Official AASIST repository and retained license: https://github.com/clovaai/aasist
3. Wang et al., *ASVspoof 2019: A large-scale public database of synthesized, converted and replayed speech*, Computer Speech & Language 64, 101114, 2020. DOI: 10.1016/j.csl.2020.101114
4. Official ASVspoof challenge distribution and protocols: https://www.asvspoof.org/
5. Baevski et al., *wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations*, 2020. https://arxiv.org/abs/2006.11477
6. Frank and Schönherr, *WaveFake: A Data Set to Facilitate Audio Deepfake Detection*, NeurIPS Datasets and Benchmarks, 2021. https://github.com/RUB-SysSec/WaveFake
7. Tak et al., *End-to-End anti-spoofing with RawNet2*, ICASSP 2021, pp. 6369–6373.

Exact project citation records and verification scope are preserved in `research/paper/references.bib`, `references_verified.md` and `reference_sources.json`. Dataset release access and licensing require independent verification at the time of use.

33. Source-of-truth guide

For implementation consult `frontend/src/main.tsx`, `backend/app.py`, `backend/schemas.py`, `ml/service.py` and `ml/policy.py`. For model identity consult `ml/constants.py`, `ml/download_model.py` and the retained vendor files. For historical verification consult `docs/VERIFICATION.md`. For measured scores consult `research/results/`. For dataset blockers consult `research/datasets/`. For research execution consult `research/README.md`, `research/validation/README.md` and `experiments/README.md`.

This document consolidates those sources and user-demonstrated behavior as of its stated date. It introduces no new benchmark, threshold, model training result or public deployment.